Python Day 4

NumPy · pandas 입문 및 미니 프로젝트

HYUNDAI AI Insight Campus · 온디바이스 AI · 프로그래밍 언어(Python) 4/4 (마지막 날)

2026. 9. 7 (월) 09:00–18:00 · 8교시 · Google Colab

NumPy | Matplotlib | pandas | 제너레이터 | 미니 프로젝트 | 팀 시연

강사: 이몽룡 · 길동

오늘의 학습 목표

수업이 끝나면 여러분은 다음을 할 수 있습니다

1

NumPy 기초

반복문 없는 벡터 연산·브로드캐스팅으로 센서 데이터를 변환한다

2

NumPy 활용

불리언 마스크·axis 통계, 이미지=H×W×C 배열을 확인한다

3

Matplotlib

시계열·히스토그램을 그리고 png 로 저장한다

4

pandas

DataFrame 필터·groupby 집계·CSV 왕복·결측 처리를 한다

5

문법 마무리

제너레이터(yield)로 수신부를 만들고 f-string 으로 표를 정렬한다

6

미니 프로젝트

수집→처리→준실시간 그래프→CSV 를 팀으로 완성·시연한다

시간표

8교시 × 50분 · 점심 12:00–13:00 · 오전 도구 3종, 오후 문법 마무리 + 팀 프로젝트

교시	시간	주제	형태	애니메이션
1	09:00–09:50	NumPy 기초 — 벡터 연산·브로드캐스팅	강의 + 실습	D4-1 ①~④
2	10:00–10:50	NumPy 활용 — 마스크·통계·이미지=배열	강의 + 실습	D4-1 ⑤~⑦
3	11:00–11:50	Matplotlib — 시계열·히스토그램	강의 + 실습	(실습 중심)
4	13:00–13:50	pandas 맛보기 — DataFrame·groupby·CSV	강의 + 실습	D4-2 ①~⑥
5	14:00–14:50	파이썬 문법 마무리 — 제너레이터·이터레이터	강의 + 실습	D3-5 재사용 (스트림=제너레이터)
6	15:00–15:50	미니 프로젝트 ① — 필수 기능	팀 실습	전체
7	16:00–16:50	미니 프로젝트 ② — 확장·시연 준비	팀 실습	전체
8	17:00–17:50	팀별 시연·코드 리뷰·정리	발표	—

오늘 사용하는 학습 도구 3종

예측 → 애니메이션에서 확인 → Colab에서 코드로 검증 → 자동 채점 — 나흘째 같은 리듬

애니메이션 7개

…/anim/index.html

- 신규 2개(D4-1·D4-2) + 재사용 5개 (day1~3)
- D4-1: 스칼라가 퍼지는 그림·마스크 골라내기
- D4-2: groupby 가 갈라졌다 합쳐지는 3단
- D3-5 재생 — "스트림은 본질적으로 제너레이터"

Colab 워크시트

Day4_worksheet.ipynb

- 설정 셀이 스트림 300줄을 자동 생성 (seed 고정)
- 1~5교시 TODO 20개 + 프로젝트 필수 자동 채점 4
- 확장 E2·E3 도 자동 채점 — 나머지는 시연으로
- 제출 요약이 out/ 산출물 4종을 확인

개념 워크시트

…/anim/worksheet.html

- 교시별 10문항 × 7 = 70문항 (Day 4 전용)
- 7번 탭은 Day 1~4 총복습 — 4일의 연결 확인
- 채점 → 오답 정리 → 캡처로 노션에
- 결과 코드를 제출 요약 셀에 입력

예측 → 애니메이션 확인 → 코드로 검증 → 자동 채점

애니메이션 — 신규 2개 + 재사용 5개

Day 4는 재사용의 날 — 3일간 본 그림이 오늘 실제 코드가 됩니다

D4-1 — NumPy 브로드캐스팅·마스크 — 1·2교시

스칼라가 퍼지는 그림 · (2,3)+(3,) · ValueError · 마스크 골라내기·집계

D4-2 — DataFrame·groupby 흐름 — 4교시

레코드→표 · 불리언 필터 · alert 열 · 분할→집계→결합 · CSV 왕복

D3-5 — 시리얼 스트림 (day3) — 5·6교시

"스트림은 본질적으로 제너레이터" — read_frames 의 명세

D3-4 — JSON 왕복 (day3) — 4교시

(선택) readings.json → DataFrame 다리

D2-5 — 트리 뷰어 (day2) — 6교시

프로젝트 데이터 구조 확인

04 — 파싱 파이프라인 (day1) — 5교시

read_frames 가 내놓는 한 줄의 처리 경로

01 — 비트마스크 (day1) — 7교시

확장 E6 — D 필드 플래그 해석

제목 클릭 시 열림 · Day 4 목차: https://devops0906.github.io/AI_CAMPUS/python-day4/anim/index.html

워크시트 사용법

Day 1~3과 동일 — 설정 셀이 데이터까지 만들어 줍니다

Colab 워크시트 — 교시당 한 섹션

1	⚙️ 설정	이름 입력 후 최우선 실행 — 스트림 300줄·parse_frame 생성
2	따라하기	실행만 하고 출력을 읽는다
3	TODO 1~4	raise NotImplementedError 를 지우고 작성
4	✅ 자동 채점	몇 번이든 재실행 — PASS n/4
5	6교시 필수	프로젝트 4항목 자동 채점 (집계·요약·CSV·png)
6	7교시 확장	E2·E3 자동 채점 — 나머지는 시연으로

개념 워크시트 (HTML · Day 4)

- 교시별 10문항 · 7번 탭은 Day 1~4 총복습
- 채점 후 정오 + 해설 + 확인할 애니메이션
- 제출 → 오답 정리 → 캡처로 노션에
- 결과 코드 7개를 제출 요약 셀에 입력

https://devops0906.github.io/AI_CAMPUS/python-day4/anim/worksheet.html

제출 (개인 + 팀)

```
이름: 홍길동
6교시-필수 자동 채점 4/4
out/readings.csv: 있음 ✅
out/sensor_summary.csv: 있음 ✅
out/timeline.png: 있음 ✅
out/hist.png: 있음 ✅
```

온라인 자료 링크

브라우저에서 바로 열립니다 — 설치·다운로드 없음

기본 주소 (Day 4)

https://devops0906.github.io/AI_CAMPUS/python-day4/anim/

index.html	Day 4 애니메이션 목차 (신규 2 + 재사용 5 링크)
worksheet.html	Day 4 개념 워크시트 (70문항 · 총복습 탭 포함)

신규 애니메이션 (day4 폴더)

1·2교시 D4-1_NumPy_브로드캐스팅_마스크.html

4교시 D4-2_DataFrame_groupby_흐름.html

재사용: D3-5 스트림 · D3-4 JSON (day3) · D2-5 트리 (day2) · 04 파이프라인 · 01 비트마스크 (day1)

1 NumPy 기초 — 벡터 연산·브로드캐스팅

09:00–09:50

학습 목표

- 리스트와 ndarray 의 차이를 설명한다 (동일 dtype·벡터 연산)
- shape / dtype 을 먼저 읽는 습관
- 반복문 없이 배열 전체를 변환한다
- 브로드캐스팅 규칙과 ValueError 메시지 읽기

🎬 D4-1 브로드캐스팅·마스크

- ① 리스트 vs 배열 — * 2 의 두 얼굴
- ② 스칼라 2 가 유령 칸으로 복제되어 퍼지는 그림
- ③ (2,3)+(3,) 행 방향 확장 → ④ 모양 불일치 ValueError
- 비교 연산(temps > 35)도 배열로 — 2교시 예고

핵심 개념

속도 대결

```
%timeit [r/1023*3.3 for r in raws]   # Day 2
%timeit big / 1023 * 3.3             # NumPy
# 반복문이 C 내부로 — 수십 배
```

두 얼굴

```
[1, 2] * 2          # [1,2,1,2] 이어 붙이기
np.array([1,2]) * 2 # [2,4]     원소별 곱
```

브로드캐스팅

```
a * 2               # 스칼라가 퍼진다
(2,3) + (3,)        # 행마다 늘어난다
(3,) + (4,)         # ValueError: shapes
```

습관

배열을 받으면 `print(a.shape, a.dtype)` 부터

1 NumPy 기초 — 실습

Colab 워크시트 TODO

1 TODO 1-1 volts
RAWS / 1023 * 3.3 — 반복문 금지

2 TODO 1-2 alert_count
(TEMPS24 > 35).sum()

3 TODO 1-3 fahr
섭씨 → 화씨 한 줄

4 TODO 1-4 브로드캐스팅 (심화)
(3,4) + (4,) 합계 · (3,) 오류 메시지 옮겨 적기

✅ **자동 채점** volts 해시 · 건수 · 화씨 · bsum (4항목)

🔍 자기 점검 (예측 → 애니메이션 → 코드)

- (3,)+(4,) → ValueError (shapes)

체크포인트 · 강사 확인

- "벡터 연산 = 반복문이 C 로" 를 자기 말로 하는가
- 리스트에 * 2 를 해서 [1,2,1,2] 를 만난 학생 — 자료형 확인
- %timeit 결과의 배율을 채팅에 공유하게 (동기 강화)

2 NumPy 활용 — 마스크·통계·이미지=배열

10:00–10:50

학습 목표

- ⚠ 슬라이스는 뷰 — 참조 모델의 재림, .copy() 방어
- 불리언 마스크로 고르고(⑥) 세고(⑦) 찾는다 (argmax)
- 2차원 axis 통계 — "axis 는 사라지는 방향"
- 이미지 = H×W(×C) 숫자 배열임을 조작으로 확인

🎬 D4-1 ⑤⑥⑦

- ⑤ 조건 → True/False 배열이 되는 순간
- ⑥ True 자리만 통과해 새 배열로 — ALERT 필터
- ⑦ mask.sum() = 건수 · argmax = 시점
- "pandas 의 df[조건] 도 같은 문법" 예고

핵심 개념

뷰 함정

```
b = a[2:5]; b[0] = 999    # a 도 바뀐다!
c = a[2:5].copy()         # 독립 사본
```

마스크

```
mask = BT > 35
BT[mask]                      # 골라내기
BT[BN == "ESP32-02"].mean()   # 노드 마스크
```

통계

```
(BT > 35).sum()    # 몇 건이
BT.argmax()        # 언제
M2.mean(axis=0)    # 행이 사라짐 → 회차별
```

이미지

```
IMG.shape  # (60, 80) — 그냥 배열
IMG[::-1] 상하 반전 · IMG * 0.5 어둡게
```

2 NumPy 활용 — 실습

Colab 워크시트 TODO

1 TODO 2-1 avgs
노드 마스크로 노드별 평균 dict

2 TODO 2-2 n_alert·idx_max
마스크 합 · argmax

3 TODO 2-3 이미지
IMG[::-1] · IMG*0.5 → imshow 3연속

4 TODO 2-4 axis (심화)
(4,6) 에서 axis=1/0 평균 + 의미 한 줄

✅ **자동 채점** avgs 참조 비교 · 건수/시점 · 반전/밝기 검사 · shape (4항목)

🔍 자기 점검 (예측 → 애니메이션 → 코드)

- 슬라이스 수정 → a 도 바뀐다 (뷰)

체크포인트 · 강사 확인

- 뷰 함정을 Day 2 참조 공유와 연결해 말하는가
- 마스크 없이 for 문으로 푼 학생 — 한 줄로 다시
- 이미지 shape 을 보고 흑백/컬러를 구분하는가

3 Matplotlib — 시계열·히스토그램

11:00–11:50

학습 목표

- 읽히는 그래프 4요소: title/xlabel/ylabel/grid (+ 범례)
- 임계선(axhline)·마스크 scatter 로 ALERT 표시
- 히스토그램으로 분포 읽기
- savefig 를 show 앞에 — 0바이트 사고 예방

🎬 **(실습 중심 — 애니메이션 없음)**

- 2교시 마스크가 그래프의 빨간 점이 되는 순간을 라이브로
- 제목에 근거(노드 수·건수) 넣는 습관
- 저장 순서를 일부러 바꿔 0바이트를 만들어 보이기
- 히스토그램 bins 를 3→12→50 으로 바꿔 감각 잡기

핵심 개념

4요소

```
plt.plot(BT, marker="o", label="T")
plt.title(f"... ({len(BT)} readings)")  # 근거
plt.xlabel · ylabel · grid · legend
```

센서 문법

```
plt.axhline(35, color="red", ls="--")
plt.scatter(np.where(mask)[0], BT[mask],
            color="red")   # 마스크 재활용
```

분포

```
plt.hist(BT, bins=12)
# 분포를 보면 이상이 보인다
```

저장

```
plt.savefig("out/x.png", dpi=150)  # ← 먼저
plt.show()                          # ← 나중
```

3 Matplotlib — 실습

Colab 워크시트 TODO

1. **TODO 3-1 timeline_boot.png**
   시계열 + 임계선 + ALERT scatter 저장
2. **TODO 3-2 hist_boot.png**
   bins=12 + 정상 범위 코멘트
3. **TODO 3-3 avg_by_node.png**
   막대 + plt.text 값 표시
4. **TODO 3-4 combo.png (심화)**
   subplots(1,2) 나란히

✅ **자동 채점** png 4종 존재·크기 (라벨은 순회 확인)

🔍 자기 점검 (예측 → 애니메이션 → 코드)

- 0바이트 원인 → show 후 savefig

체크포인트 · 강사 확인

- 4요소가 반사적으로 들어가는가
- 0바이트 png 학생 — 순서 교정 후 재저장
- 그래프만 보고 ESP32-02 이상 구간을 지목하는가

4 pandas 맛보기 — DataFrame·groupby·CSV

13:00–13:50

학습 목표

- DataFrame = dict 레코드 목록의 완성형 (행=레코드)
- head/info/describe 3종 세트 습관
- 불리언 필터 = NumPy 마스크와 같은 문법
- groupby 분할→집계→결합 · CSV 왕복 · 결측 처리

🎬 D4-2 DataFrame·groupby

- ① 레코드 목록 → 표 → ② 열 선택
- ③ 불리언 필터 — 행이 걸러지는 장면 → ④ alert 열
- ⑤ 분할→집계→결합 3단 애니메이션
- ⑥ CSV 왕복 — index=False 유령 열 경고

핵심 개념

3종 세트

```
df = pd.DataFrame(RECORDS_BOOT)
df.head()  · df.info()  · df.describe()
# info 로 T 의 dtype 부터 확인!
```

고르기

```
df["alert"] = df["T"] > 35    # 새 열
df[df["alert"]]                # 행 필터
df.sort_values("T", ascending=False)
```

groupby

```
df.groupby("node")["T"]
  .agg(["mean", "max", "count"])
# stats_by_node 3일의 진화 — 한 줄
```

왕복·결측

```
to_csv(index=False) ↔ read_csv  · isna().sum() → dropna
```

4 pandas — 실습

Colab 워크시트 TODO

1. **TODO 4-1 df 3종 세트**
   26행 4열 + n_rows
2. **TODO 4-2 alert·hot**
   불리언 열 + 필터 + 내림차순
3. **TODO 4-3 groupby 왕복**
   요약 → to_csv → read_csv
4. **TODO 4-4 결측 (심화)**
   isna().sum()==2 · dropna 후 3행

✅ **자동 채점** shape · 필터 정합 · 요약 equals + 왕복 · 결측 (4항목)

🔍 자기 점검 (예측 → 애니메이션 → 코드)

- 전부 False → T 가 문자열(object)

체크포인트 · 강사 확인

- df[조건] 을 NumPy 마스크와 연결해 말하는가
- groupby 결과와 Day 3 stats_by_node 출력 비교시키기
- to_csv 에 index=False 를 빼먹은 학생 — 유령 열 관찰

5 파이썬 문법 마무리 — 제너레이터·이터레이터

14:00–14:50

학습 목표

- enumerate/zip — range(len()) 순회 졸업
- 이터레이터는 한 번 소모 (map 회수)
- 제너레이터(yield)로 read_frames 수신부 완성
- 제너레이터 표현식 · f-string 정렬 (:10s :8.2f)

🎬 D3-5 재사용 · 04

- D3-5 ⑤ 재생 — "readline 이 yield 였던 것"
- 호출해도 안 돌고, for 가 꺼낼 때 도는 것 확인
- 재소모 sum → 0 실험 — 소모 성질 각인
- 04: read_frames 가 내놓는 한 줄의 처리 경로

핵심 개념

enumerate·zip

```
for i, line in enumerate(f, 1): ...
dict(zip(nodes, avgs))    # 쌍 → dict
```

한 번 소모

```
m = map(str, [1,2])
list(m)   # ['1','2']
list(m)   # [] — 스트림과 같은 성질
```

제너레이터

```
def read_frames(path):
    for line in f:
        ...
        yield parse_frame(line)   # 멈췄다 이어서
# 100만 줄도 리스트 없이
```

정렬 표

```
f"{name:10s}{avg:8.2f}{cnt:7d}"   # report 가 반듯
```

5 제너레이터·문법 — 실습

Colab 워크시트 TODO

1. **TODO 5-1 bad_lines**
   enumerate 로 손상 줄 번호 3개

2. **TODO 5-2 read_frames**
   yield · first/rest 25/재소모 0

3. **TODO 5-3 f-string 표**
   10s / 8.2f / 7d 정렬

4. **TODO 5-4 avg_gen (심화)**
   제너레이터 표현식 — 리스트 없는 평균

✅ **자동 채점** 줄 번호 · isgeneratorfunction+소모 검사 · 문자열 일치 · 평균 (4항목)

🔍 자기 점검 (예측 → 애니메이션 → 코드)

- 재소모 → 아무것도 나오지 않는다

체크포인트 · 강사 확인

- "스트림=제너레이터" 연결을 자기 말로 하는가
- read_frames 안에서 return 을 쓴 학생 — yield 와 대조
- 이 함수가 6교시 수신부로 그대로 승격됨을 공지

6 미니 프로젝트 ① — 필수 기능

15:00–15:50 · 2인 1팀

"센서 데이터 수집·시각화 도구"

```
stream_day4.txt ─▶ read_frames() ─▶ 레코드 축적
  sensor_summary.csv ◀─ groupby ◀─ DataFrame
  timeline.png ◀─ matplotlib(임계선·범례)
  (준실시간) 50건마다 clear_output 갱신
```

오전의 부품이 전부 조립됩니다

- 수신 = 5교시 read_frames
- 처리 = 1·2교시 NumPy + 4교시 pandas
- 그래프 = 3교시 + clear_output

필수 5항목 (워크시트 6교시 — 자동 채점 4)

1	수신·집계 collect() — good/bad/skip 정답 일치	✅ 자동
2	준실시간 — 50건마다 clear_output + 최신 200건	시연 확인
3	DataFrame·groupby 요약 (mean/max/count) 정합	✅ 자동
4	readings.csv + sensor_summary.csv 왕복	✅ 자동
5	timeline.png(3노드 범례+임계선) · hist.png	✅ 존재 + 라벨 순회

전원 목표: 이 교시 안에 필수 1·3·4 완주 — 2·5 는 7교시 전반

7 미니 프로젝트 ② — 확장·시연 준비

16:00–16:50

확장 메뉴 — 팀당 1개 이상 (E2·E3 는 자동 채점 제공)

E1	히스토그램 노드별 겹치기(alpha) + 코멘트	★
E2	손상 유형 — Counter 로 FrameError/ValueError 분리 ✅	★
E3	이동 평균 rolling(5).mean() 겹치기 ✅	★★
E4	열 이벤트 탐지 — ESP32-02 마스크로 시작·끝 인덱스	★★
E5	임계값 파라미터화 — 33/35 비교 표	★★
E6	D 필드 비트 플래그(문/창/모션) — Day 1 회수	★★★

시연 형식 (팀당 3분)

1분	준실시간 수집 데모 — 리허설 1회 필수, 시연 중 첫 실행 금지
1분	요약표·그래프 — "어느 노드가 언제 문제였나"
1분	코드 하이라이트 1곳 + 배운 것/막힌 것 1개

막히면

- 마스크·집계 → D4-1 ⑤~⑦
- groupby → D4-2 ⑤
- 수신 집계 불일치 → D3-5 ③④
- E6 비트 → 01 비트마스크

발표 자료 별도 제작 금지 — 코드와 산출물이 곧 발표 자료입니다

8 팀별 시연 · 코드 리뷰 · 정리

17:00–17:50

30분	**팀별 시연 3분 × 최대 8팀**	시연 후 다른 팀이 "훔치고 싶은 한 줄" 지목
7분	**강사 코드 리뷰**	잘한 패턴 3개(화면 공유) + 공통 함정 2개
9분	**4일 총정리 + C/C++ 연결**	다음 슬라이드 2장으로
4분	**제출 마감 확인·수료 인사**	팀 out/ 4종 + 개인 노트북

평가 루브릭 (Day 4 몫)

필수 5항목	자동 3×12 + 준실시간 8 + 그래프 6	50
확장 1개 이상	동작 15 + 결과물 반영 5	20
코드 품질	함수 분리·이름·중복 제거	15
시연·리뷰 참여	데모 성공 10 + "한 줄" 기여 5	15

제출: Drive › Day4 › 제출 › 팀이름/ (out 4종+코드+README)
개인: Day4_이름.ipynb (제출 요약 실행)
개념 워크시트:
…/anim/worksheet.html

오늘의 핵심 정리

각 항목 옆 번호는 다시 볼 애니메이션

벡터 연산	반복문이 C 내부로 — 그래서 빠르다 (%timeit 이 증인)	D4-1
마스크	조건이 데이터가 된다 — 고르고(⑥) 세고(⑦) 찾는다(argmax)	D4-1
그래프	그래프는 주장, 제목은 근거 — savefig 는 show 앞에	—
DataFrame	list of dict 의 완성형 — df[조건] 은 마스크와 같은 문법	D4-2
groupby	분할→집계→결합 — stats_by_node 3일의 진화가 한 줄로	D4-2
제너레이터	yield 로 멈췄다 이어서 — 스트림은 본질적으로 제너레이터	D3-5

다음 과목·장비 주간으로 가는 다리

여러분은 이미 절반을 알고 있습니다

C / C++ 과목

NumPy 가 빨랐던 이유 = 안쪽이 C. 이제 그 안쪽으로 내려갑니다.

포인터	= Day 2 참조 모델 (02) 의 C 버전
구조체 `struct`	= Day 3 클래스의 C 버전
`printf` 포맷	= 오늘 f-string 정렬의 조상
배열과 메모리	= 오늘 ndarray 가 실제로 놓이는 곳

장비 실습 주간 (Jetson·RPi)

```
# 오늘의 코드
with open("data/stream.txt") as f: ...
# 장비 주간 — 한 줄 치환 (라이브 시연)
ser = serial.Serial("/dev/ttyUSB0", 9600)
```

- read_frames 의 open 한 줄이 Serial 로 — 나머지는 그대로
- pyserial·GPIO 는 장비 주간 첫 시간에 실행
- OpenCV 카메라 프레임 = 오늘의 H×W×C 배열
- 오늘 팀 프로젝트 코드가 장비 실습의 출발 코드

Python 4일 완주

변수 하나에서 — 저장되고, 집계되고, 그려지는 시스템까지

Day 1	변수·비트·제어 — 값 하나를 다루는 법
Day 2	자료구조·함수 — 담고, 다듬는 법
Day 3	클래스·파일 — 묶고, 남기는 법
Day 4	NumPy·pandas·프로젝트 — 크게 다루고, 보여주는 법

오늘 잊지 말 것

- 팀 out/ 4종 제출
- 개인 노트북 제출
- 개념 워크시트 캡처

수고하셨습니다. 다음 주, 여러분의 코드가 진짜 센서를 만납니다.